

Design to code: handoff, tokens, and shipping it for real

Week 10, Lecture 2 · Design for Builders: Ship Beautiful Products as a Founder

Autumn 2026

What we will cover

- What design tokens are and how they map from Figma to CSS
- The Figma-to-Tailwind pipeline in concrete steps
- Writing a component spec a developer can implement
- What lives in design vs what lives in code

Part 1: what design tokens actually are

The three-layer model

Design tokens have three layers. Confusing them is the most common handoff mistake.

Layer	What it contains	Example
Primitive	Raw values, no semantic meaning	<code>blue-500: #3b82f6</code>
Semantic	Use-case names, pointing to primitives	<code>color-action-primary: {blue-500}</code>
Component	Component-specific names, pointing to semantic	<code>button-background-default: {color-action-primary}</code>

Figma's variable collections implement exactly this hierarchy. If your file collapses all three into one flat list, the handoff will be painful.

Why this matters for code

A primitive token exported as a CSS custom property:

```
--blue-500: #3b82f6;
```

A semantic token that references it:

```
--color-action-primary: var(--blue-500);
```

A Tailwind config that uses the semantic token:

```
// tailwind.config.js
colors: {
  "action-primary": "var(--color-action-primary)",
}
```

When you change `--blue-500`, everything downstream updates automatically. This is what makes a token system worth building

Figma variables and CSS variables are the same idea

Figma (2024): "Design tokens are the single source of truth that connect design and code."

The mapping is direct:

- A Figma variable collection is a `:root` block
- A Figma variable is a CSS custom property
- A Figma variable mode (light/dark) is a media query or data attribute

If you named your Figma variables with slashes, they become kebab-case custom properties:

`color/brand/500` becomes `--color-brand-500`.

Part 2: the Figma-to-Tailwind pipeline

The four steps

1. Define primitive and semantic collections in Figma variables
2. Export to JSON via the Figma Variables REST API or Tokens Studio plugin
3. Transform the JSON to CSS custom properties with Style Dictionary
4. Configure Tailwind to reference those custom properties

anclist.com (2024) walks this pipeline end to end for a Tailwind project. The pipeline takes about thirty minutes to set up the first time. After that, updating a token in Figma and propagating it to your codebase is a single export step.

Step 1: Figma variables, structured correctly

Two collections minimum:

- `primitives`: raw color values, raw spacing values (named by scale position)
- `semantic`: use-case names that reference primitives

```
primitives/  
  color/  
    blue-50 → #eff6ff  
    blue-500 → #3b82f6  
    blue-900 → #1e3a5f  
  space/  
    1 → 4px  
    2 → 8px  
    4 → 16px  
  
semantic/  
  color/  
    surface/base → {primitives/color/slate-50}
```

Step 2: exporting to JSON

Two options:

Figma Variables REST API (free): Available to anyone with the file. Returns a JSON object with variable collections, modes, and values. Requires a personal access token and a GET request.

Tokens Studio plugin (free tier): GUI-based. Exports to a JSON file in W3C Design Tokens format. Easier to set up for a one-person project.

For a solo founder shipping their own product, Tokens Studio is faster. For a team, the REST API is more repeatable.

Step 3: Style Dictionary

Style Dictionary (Amazon, open source) takes a JSON token file and outputs CSS custom properties, Tailwind config, Swift, Kotlin, or any other format you configure.

```
npx style-dictionary build
```

Output for CSS:

```
:root {  
  --color-action-primary: #3b82f6;  
  --color-surface-base: #f8fafc;  
  --space-1: 4px;  
  --space-2: 8px;  
  --space-4: 16px;  
}
```

You commit this file to your repo. Tailwind reads it.

Step 4: Tailwind config

```
// tailwind.config.js
module.exports = {
  theme: {
    extend: {
      colors: {
        "action-primary": "var(--color-action-primary)",
        "surface-base": "var(--color-surface-base)",
      },
      spacing: {
        "1": "var(--space-1)",
        "2": "var(--space-2)",
        "4": "var(--space-4)",
      },
    },
  },
};
```

Now `bg-action-primary` and `p-4` in your Tailwind classes are driven by tokens from your

Part 3: writing a component spec

What a component spec contains

A spec answers one question for the developer: "What do I need to know to implement this component without opening Figma again?"

Required sections:

1. Component name and description (one sentence)
2. Variants and states (list all combinations)
3. Token references (which token for each visual property)
4. Anatomy (which sub-elements exist and how they relate)
5. Responsive behavior (what changes at each breakpoint)
6. Accessibility notes (ARIA role, keyboard behavior, focus ring)

A button spec, abbreviated

Button / Primary

States: default, hover, pressed, disabled, loading

State	Background	Text	Border
Default	<code>--color-action-primary</code>	<code>--color-text-on-action</code>	none
Hover	<code>--color-action-primary-hover</code>	<code>--color-text-on-action</code>	none
Disabled	<code>--color-action-disabled</code>	<code>--color-text-disabled</code>	none

Spacing: padding-inline 16px (`--space-4`), padding-block 8px (`--space-2`), gap between icon and label 8px.

Typography: `--type-label-md` (14px / 1.2 / 500 weight).

ARIA: `role="button"`, `aria-disabled="true"` when disabled (not `disabled` attribute alone).

What Figma Dev Mode gives you for free

When your Figma file is structured correctly, Dev Mode surfaces:

- All applied styles as CSS property names
- Token references by name (if you used variables)
- Spacing and size values derived from your grid
- Export settings for assets

A well-structured Figma file reduces the spec to the things Dev Mode cannot tell the developer: interaction states, animation timing, accessibility behavior, and business logic.

Part 4: what lives in design vs what lives in code

The boundary

Lives in design

- Visual appearance (color, type, spacing, shape)
- Layout and composition
- States and their visual treatment
- Asset exports (icons, illustrations, images)
- Token names and their values

Lives in code

- Interaction behavior (what triggers a state change)
- Animation timing
- Business logic (when to show an error vs a

The gray zone

Some things sit on the boundary and require a conversation:

- Motion: duration and easing can be tokens, but the trigger is code
- Responsive layout: breakpoints are tokens, but the layout logic is code
- Empty states: the visual design is yours, but the condition that shows it is the engineer's

Name the gray zone explicitly in your spec. "This component shows the loading state when the data fetch takes longer than 300ms. That threshold is set in code, not design." That one sentence prevents a week of back-and-forth.

The simplest possible handoff for a solo founder

If you are implementing your own design, the token pipeline is overkill for a first launch. Do this instead:

1. In Figma, open your color variables and write down every semantic token name and value
2. In your CSS, add them to `:root` by hand
3. In Tailwind config, reference them by CSS variable name
4. Document one text frame in Figma: "Token changes go in `:root` in `globals.css`"

The full export pipeline is the right answer when you have a team, or when you are updating tokens more than once a month. For a solo first launch, hand-sync is fine.

Take-aways

1. Design tokens have three layers: primitive (raw values), semantic (use-case names), and component (component-specific names). Get the hierarchy right before you export anything.
2. The Figma-to-Tailwind pipeline is four steps: define in Figma, export to JSON, transform with Style Dictionary, configure Tailwind. anclist.com (2024) walks every step for a Tailwind project.
3. A component spec must tell the developer every state, every token reference, and every accessibility behavior. Dev Mode handles the rest.
4. The boundary between design and code is mostly clear (appearance vs behavior) but has a gray zone. Name the gray zone explicitly in the spec.
5. For a solo first launch, hand-syncing tokens to CSS variables is fast enough. Use the full export pipeline when you have a team or frequent token changes.



A token is just a named value. The name is the design decision. The value is just a number.

, Figma, Design Tokens resource (2024)

What's next

- **Section this week:** Mutual critique. Bring your complete launch package. Two structured peer reviews.
- **Reading:** Week 10 reading: the course spine, design-to-code handoff summary, and a post-course practice plan.
- **CAPSTONE due:** Launch package submitted before section.
- **After the course:** Your design system file is a living document. Keep it. Your tokens, your component library, and your brand sheet are the three artifacts that will compound in value as your product grows.